

Regularization Methods for Robust Neural Networks

Overview

- Problem: Neural networks with many parameters often overfit (perform well on training data but poorly on unseen data).
- Solution: Regularization → set of techniques to prevent overfitting and improve generalization.



Regularization for Neural Networks



Overfitting

High training accuracy, low generalization



Apply Regularization

Add penalty for complex models



Improved Generalization

Better performance on unseen data

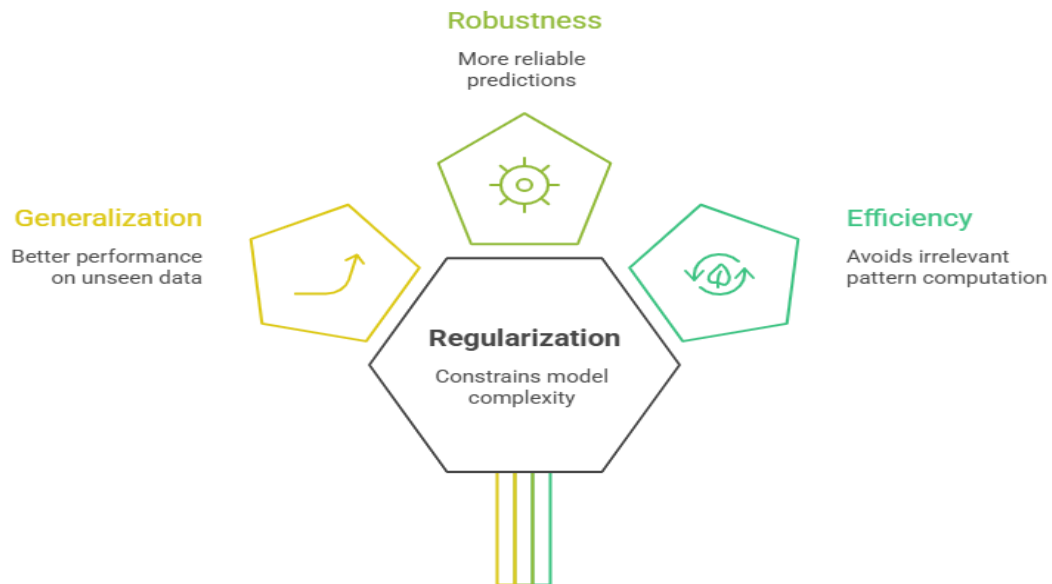
Analogy:

Think of a student memorizing answers (overfitting) vs truly understanding concepts (generalization).

Regularization acts like discipline rules that stop the student from just memorizing and force them to think properly.

Why Do We Need Regularization?

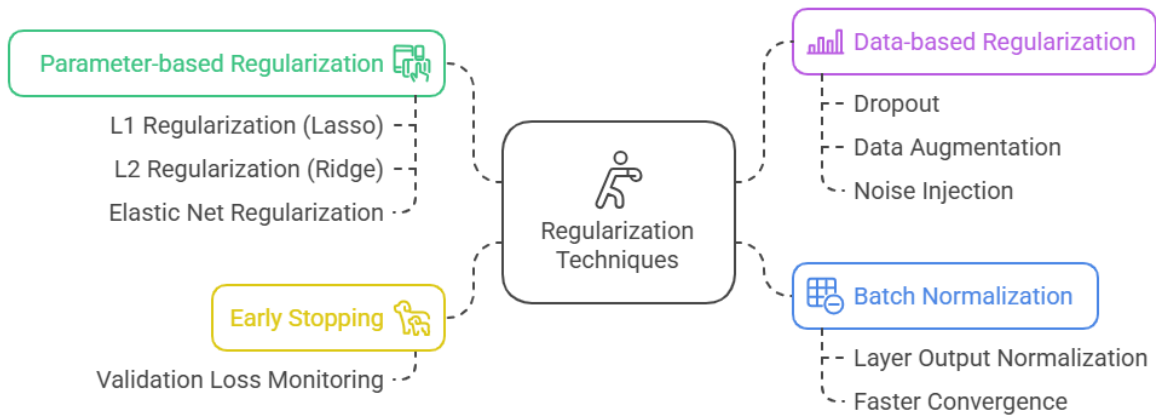
Regularization Improves Model Generalization



- Models with too much capacity (deep/wide networks) can memorize training data.
- Overfitting leads to poor test accuracy.
- Regularization ensures:
 - Better generalization.
 - More robust predictions.
 - Avoids wasted computation on irrelevant patterns.

Types of Regularization

Types of Regularization Techniques



A. Parameter-based Regularization

1. L1 Regularization (Lasso)

$$\text{Loss} = \text{Original Loss} + \lambda \sum |w_i|$$

Loss = Original Loss + \lambda \sum |w_i|

- Adds penalty proportional to absolute values of weights.
- Encourages sparsity (many weights become zero).
- Useful for feature selection.

Example: If you train on text data, L1 will automatically drop irrelevant words.

2. L2 Regularization (Ridge)

$\text{Loss} = \text{Original Loss} + \lambda \sum w_i^2$
 $\text{Loss} = \text{Original Loss} + \lambda \sum w_i^2$

- Adds penalty proportional to squared values of weights.
- Encourages smaller weights (but not zero).
- Helps prevent very large weights that cause unstable models.

Example: Used in logistic regression, linear regression, and almost every deep learning model (via weight decay).

3. Elastic Net Regularization

- Combines L1 + L2 penalties.
- Useful when data has correlated features.

B. Data-based Regularization

1. Dropout

- Randomly “drops out” (sets to 0) some neurons during training.
- Prevents co-adaptation of neurons.
- At test time → full network is used.

Example: If dropout = 0.5 → half neurons are ignored in each iteration.

2. Data Augmentation

- Artificially expands training data by making modified copies.
- In vision tasks: rotations, flips, cropping, color shifts.
- In NLP: synonym replacement, back-translation.

Example: A cat image can be flipped or rotated → still a cat → helps model generalize.

3. Noise Injection

- Adding small random noise to inputs, weights, or outputs.
- Makes the model robust to slight variations.

Example: Adding Gaussian noise to speech signals to make speech recognition robust.

C. Early Stopping

- Stop training when validation loss stops improving.
- Prevents over-training (memorization).

Example: Model trains for 100 epochs, but validation accuracy peaks at 40 epochs → we stop at 40.

D. Batch Normalization (BN) (sometimes considered regularization)

- Normalizes layer outputs.
- Adds slight noise during training → acts as a regularizer.
- Also helps with faster convergence.

Applications in Deep Learning

Deep Learning Applications					
Application	L1/L2	Dropout	Data Augmentation	Early Stopping	BatchNorm
Deep Learning Models	Yes	Deep CNNs, RNNs	Computer vision, NLP	Yes	Yes

- L1/L2 → Almost all models (weight regularization).
- Dropout → Deep CNNs, RNNs.
- Data Augmentation → Computer vision, NLP.
- Early Stopping → Training control in any DL task.
- BatchNorm → Faster + stable training in deep networks.

Key Takeaways

- Overfitting is a major issue in deep learning.
- Regularization helps control complexity and improve generalization.
- Popular methods: L1, L2, Dropout, Data Augmentation, Early Stopping, BatchNorm.
- Choice depends on task:
 - Vision → Dropout + Data Augmentation.
 - NLP → L2 + Dropout.
 - Tabular data → L1/L2.

